

Full-Stack Engineer Assignment

Product Listing Page (PLP)

Company: Appscrip | **Role:** Full-Stack Engineer (AI-Native)

1. Overview

You will build a **Product Listing Page (PLP)** for an e-commerce storefront, end to end. This covers the UI from the Figma design, a backend API that serves the product data, a database behind it, and a deployed, publicly shareable app.

This is not just a UI task. We want to see how you structure a full-stack project, design clean APIs, render on the server, and ship something production-minded.

2. Design Reference

Figma file: <https://www.figma.com/file/NOTv7yYlf3kfMLQjUncUlx/Design-Task---PLP?type=design&node-id=0-1&mode=design&t=mEpvVYQ7GlnQPxxk-0>

Follow the design closely for layout, spacing, typography, colors, and components. Where the design doesn't cover something (tablet layout, empty states, loading states, error states), use your own judgment and keep it consistent with the design language.

3. What You Need to Build

A. Frontend

1. Implement the page first in **plain HTML and CSS** (a static version of the design), then convert it into a functional app.
2. Build the functional page in **React with Next.js** (App Router preferred), or **Vue with Nuxt.js**. Use TypeScript if you can.
3. The page must include:
 - Header/navigation and footer as per the design
 - Product grid with product cards (image, title, price, and other design elements)
 - Filters and sorting as shown in the design (e.g., category, price, sort by)
 - Pagination or infinite scroll
 - Loading, empty, and error states
4. **Responsive** for desktop, tablet, and mobile. No horizontal scroll at any breakpoint.
5. **SSR is mandatory.** The initial product list must be server-rendered, so "view source" shows real product content. Filter/sort state should be reflected in the URL (query params) so pages are shareable and crawlable.

B. Backend

Build your own backend API using **Node.js** (**NestJS** preferred; Express or Fastify with a clean structure is acceptable).

1. Expose REST endpoints at minimum:
 - GET /products: supports pagination (page, limit), filtering (category, price range), sorting (sort), and search (q)
 - GET /products/:id: single product
 - GET /categories: list of categories
2. Validate all inputs (DTOs/schema validation) and return consistent error responses with correct HTTP status codes.
3. Add basic API documentation (Swagger/OpenAPI or a clear README section).
4. Handle CORS, environment config, and secrets properly (no secrets committed).

C. Database

1. Use **PostgreSQL** (preferred) with an ORM such as Prisma or Drizzle.
2. Design a sensible schema (products, categories, images, etc.) with migrations.
3. Add indexes where they make sense for your filter and sort queries.
4. Include a **seed script** that populates the DB. You may seed from <https://fakestoreapi.com/> data, but the app must read from **your own API and DB**, not call FakeStore directly from the frontend.

D. Integration

- The Next.js/Nuxt server fetches from your backend during SSR.
- Client-side interactions (filtering, sorting, pagination) should update the URL and fetch data efficiently without a full reload.

4. SEO Requirements

- Unique, meaningful **page title** and **meta description**
- Correct heading structure: a single **H1**, with logical **H2s** (and **H3s** where needed)
- **Schema.org structured data** (JSON-LD), e.g., ItemList and Product for listed items, and BreadcrumbList if applicable
- Images with **SEO-friendly file names** (e.g., mens-cotton-slim-fit-tshirt.jpg, not IMG_001.jpg) and meaningful **alt text**
- Semantic HTML, canonical URL, and Open Graph tags
- Optimized images (correct sizing, lazy loading below the fold, modern formats where possible)

5. Performance and Quality Expectations

- Keep the **DOM size minimal**: no unnecessary wrapper elements
- Use **minimum pre-built JS packages**. Justify any dependency you add in the README. Don't pull in a UI kit or heavy libraries for things you can build easily.
- Good Lighthouse scores are a plus (Performance, Accessibility, SEO, Best Practices)
- Accessibility basics: semantic elements, keyboard navigation, focus states, sufficient contrast, ARIA where appropriate
- Clean, consistent **naming conventions** and a clear **folder structure** on both frontend and backend

6. Use of AI Tools

AI coding tools (Cursor, Claude Code, Copilot, etc.) are **allowed and encouraged**. We use them daily. However:

- You are responsible for every line you submit. You should be able to explain any part of your code in the follow-up discussion.
- In your README, add a short "**AI usage**" section: which tools you used, where they helped, and one example where you had to correct or reject AI output.
- If you include project context files for AI tools (e.g., AGENTS.md, CLAUDE.md, or Cursor rules), keep them in the repo. It's a plus.

7. Hosting and Submission

Hosting (both must be publicly accessible)

- **Frontend:** Netlify (or Vercel). Make sure SSR actually works in the deployed version.
- **Backend and DB:** any public service (Render, Railway, Fly.io, Supabase, Neon, etc.)

GitHub

- Public repository named ``Appscrip-task-<your-name>`` (e.g., Appscrip-task-JohnDoe). A monorepo (/frontend, /backend) is fine, or use two repos with the same naming pattern.
- Commit regularly with meaningful messages. We will read your commit history.

README must include

1. Live frontend URL and live API URL
2. Tech stack and why you chose it
3. Setup steps to run locally (frontend, backend, DB, seed)
4. Architecture overview and folder structure
5. API endpoint list (or link to Swagger docs)
6. SSR approach and SEO decisions
7. Dependencies used and why
8. AI usage section (see above)
9. Known limitations and what you'd improve with more time

Send us: the GitHub repo link, the live frontend link, and the live API/docs link.

8. How We Will Evaluate You

Area	What we look for	Weight
Code structure and naming	Clear folder structure, separation of concerns, consistent naming, readable code	20%
Backend and database	API design, validation, schema and indexing, error handling, seed and migrations	20%
Frontend and design fidelity	Match to Figma, component quality, state handling, working filters/sort/pagination	15%
SSR and performance	Real server rendering, minimal DOM, minimal dependencies, loading behavior	15%

Area	What we look for	Weight
SEO	Title, description, H1/H2, schema, image names, alt text	10%
Responsiveness	Mobile, tablet, desktop fit with no layout breaks	10%
Deployment and documentation	Working public links, clear README, easy to run locally	5%
AI usage and judgment	Thoughtful use of AI, honest reflection, ability to explain the code	5%

We believe you have great potential. All the best!